

Yongjun Rong

Cherry Hill, NJ, USA, 08003 (US Citizen) **Email:** rongyj@hotmail.com **Tel:** 856-408-4027

Linkedin: <https://www.linkedin.com/in/rongyongjun/>, **Github:** <https://github.com/rongyj>

EDUCATION

Jun.,2006, Master of Computer Science, Texas Tech University, Lubbock, TX, USA

Feb.,1996, Master of Engineering, Shanghai JiaoTong University, Shanghai, China

Jul.,1991, Bachelor of Engineering, Changsha Railway University, Changsha, Hunan, China

SUMMARY:

Platform architect and hands-on engineer who builds the shared infrastructure AI applications run on. Four years delivering production GenAI systems - RAG over ChromaDB/Milvus, multi-agent orchestration with AutoGen, prompt engineering and tuning, concurrent LLM invocation, model-backed services behind FastAPI - on top of fifteen years of Infrastructure as Code and multi-tenant AWS container platforms (ECS/EKS, Istio service mesh with mesh-wide mTLS, versioned Terraform/Terragrunt module libraries). At McKinsey, built the GenAI platform capability that accelerated mainframe modernization for enterprise clients: COBOL to Java and BMS/JSF/JSP to React through multi-agent pipelines, plus LLM-generated PySpark/Databricks pipeline code shipped with generated unit tests. Now owns the reference architecture for an on-demand application platform serving hundreds of developers at a global bank. Sets the team's AI-assisted development practice there, and packages recurring platform work as agent skills and subagents behind an MCP server. Brings the governance half as well: authored group-wide architecture handbooks, set invest/divest positions on ~200 technologies, and architected a commercial multi-cloud cost analytics and optimization platform - directly transferable to token accounting, cost attribution and quota enforcement for shared AI infrastructure. US Citizen; fluent English, native Chinese.

SKILLS:

- GenAI platform: RAG over ChromaDB and Milvus - corpus curation, OpenAI embeddings, HNSW-indexed collections, chunk and top-k tuning, embedding refresh on source change; multi-agent orchestration (AutoGen); prompt engineering; concurrent async LLM invocation; model-backed APIs on FastAPI.
- Agentic automation: agent skills and subagents for recurring engineering work; MCP servers exposing platform APIs to coding agents; multi-agent pipelines with hand-off contracts, retry, escalation and human-review gates.
- AI-assisted development: Claude Code, GitHub Copilot and Cursor daily; team practice for what generated code is trusted with and the review it has to pass.
- AI-assisted modernization: COBOL to Java and BMS/JSF/JSP to React through multi-agent and RAG pipelines; LLM-generated PySpark/Databricks code shipped with generated unit tests.
- AWS platform architecture: multi-tenant ECS/EKS, Istio with mesh-wide mTLS, serverless event-driven systems (EventBridge, Lambda, Step Functions, SQS/SNS), self-service environment provisioning. Azure AKS and GCP exposure.
- Infrastructure as code: Terraform and Terragrunt module libraries, DRY multi-environment configuration, IaC-provisioned EKS, IAM/OIDC/IRSA, Route 53, ACM, RDS, secrets; Puppet, Chef, Ansible.
- Security, observability and cost: Secrets Manager and KMS, least-privilege IAM, per-tenant quotas, TLS/mTLS; Splunk, Dynatrace, Prometheus/Grafana/Jaeger/Kiali, CloudWatch; FinOps allocation, showback, unit-cost metrics, rightsizing.
- Distributed systems and languages: event-driven architecture, CQRS, event sourcing, Kafka streams, design-first microservices on OpenAPI, Databricks/PySpark; Python, Java/Spring Boot, TypeScript/Node.js, Terraform, Bash.
- Technical leadership: group-wide architecture handbooks at Barclays, a ~30-member technology strategy working group, a ~500-application cloud migration readiness assessment; mentors engineers on IaC, service mesh, prompt engineering and agentic workflows.

HOBBIES

- Pickleball (~4.0 player)
- IoT or Robot programming (Have iRobot and MIPOSaur at home. Programming against iRobot create SDK and MIPOSaur SDK.)
- Playing with cutting-edge technologies (genAI/Autogen/Multiagents, Docker, Go, Kafka, Kubernetes and Amazon CloudFormation, JavaScript, AngularJS, ReactiveJS, Node.js)

WORK EXPERIENCE SUMMARY:

Employments	Major Accomplishments & Technologies
Oct.,2025 – Current Intuitive.ai Cherry Hill, NJ (WFH),USA Principal Cloud and Platform Architect (JPMC)	- On Demand Environment (ODE) backend (Java Spring Boot) infrastructure (ECS) design and implementation - Design and implement Splunk and Dynatrace onboarding for Spring Boot ECS Microservices - Design and implement AWS Secrets (ASM) solution for Java Spring Boot Microservices - EKS Service Mesh (Istio) on-demand framework Design and Implementation - Agentic automation of platform operations: MCP server, agent skills and subagents IaC, Terraform, AWS/EKS/Service Mesh/Istio, TLS Certificate, AI-assisted development (Claude Code, GitHub Copilot, Cursor), AI agents / MCP / agentic workflow automation,
Feb.,2025 – Oct.,2025 Biophy Cherry Hill, NJ (WFH),USA Director of Cloud Architecture	- Comprehensive Infrastructure as Code (IaC) Design and Implementation - AI-Powered Data Ingestion and Scraping System for RAG IaC, Terraform/Terragrunt, AWS/EC2/EBS/S3/VPC/ALB/Security/R53/IAM/Secrets Manager/Lambda/CloudWatch/EventBridge/SQS/SNS, Python/Django/Pydantic, openAI API/LangChain, SOLID Principles, Clean Architecture,
Dec.,2021 – Feb.,2025 McKinsey & Company Cherry Hill, NJ (WFH),USA Senior Cloud Engineer II (Associate)	- Using genAI (openAI) with RAG (ChromaDB, Milvus) to accelerate the development of data pipelines running on pyspark and databricks - Using genAI (openAI) with multi-agents to accelerate the migration of mainframe to modern technologies - Migration/Refactoring of Mainframe (IBM MQ) EDA system to AWS by using EventBridge/Lambda/Step Function/CloudWatch/EKS/SQS/SNS...etc. genAI/openAI/Langchain/RAG/Multi-agents/prompts tuning, VectorStore/ChromaDB/Milvus, Python/Flask/FastAPI/Pydantic/Jupyter, Databricks/pyspark, Typescript/JavaScript/NodeJs, Event Driven Architecture (EDA), Terraform, Microservices, Java/SpringBoot/NodeJs/Python/Flask/FastAPI, EventBridge/CloudWatch/Lambda/Step Functions, DynamoDB/RDS/Oracle, Docker/K8s/EKS/AKS, Json Schema, JOI, OpenAPI/Swagger, AI-assisted development (GitHub Copilot, Cursor),
Dec.,2020 – Dec., 2021 Yotascale Cherry Hill, NJ (WFH),USA Staff Engineer	Lead Architect and Lead Implementation Engineer for Cloud(AWS/Azure/GCP) Cost Analytics/Optimization/Management platform OpenAPI/Swagger, Microservice, Java/Kotlin/SpringBoot, Kubernetes/EKS/AKS, Istio, Terraform, CQRS, Event Store, Event Driven Architecture (EDA), Kafka/ActiveMQ/IBM MQ, Kafka Stream, Docker, AWS(EC2/IAM/S3/ECS/EKS/ELB/Lambda/RDS/VPC/R53/Secret Manager/AutoScaling/SES..etc), Puppet/Chef/Ansible, Git/Maven/Jenkins, Python/Bash/Go, ELK/AppDynamic/Datadog,
Mar.,2020 – Nov., 2020 Gigster Inc. Cherry Hill, NJ,USA Technical Infrastructure/Devops Architect	AWS EKS platform for Node.js microservices and event-driven workloads Microservice, NodeJs, Kubernetes/EKS, Terraform, RabbitMQ/ActiveMQ, Docker, AWS(EC2/IAM/S3/EKS/ALB/RDS/VPC/R53/AutoScaling/SES/ECR..etc), Puppet/Chef/Ansible, Git/Maven/Jenkins, JavaScript,
Jun.,2017 – Feb., 2020 Barclays Wilmington, DE,USA VP,Lead Architect Engineer	- Delivery Lead for strategic distributed system technology domain - Group-wide Cloud Adoption for Contact Center, Merchant Services and Fraud Terraform, CQRS, Event Store, Event Driven Architecture (EDA), Kafka/ActiveMQ/IBM MQ, Kafka Stream, Docker, AWS(EC2/IAM/S3/ECS/EKS/ELB/Lambda/RDS/VPC/R53/Secret Manager/AutoScaling/SES..etc), Puppet/Chef/Ansible, Git/Maven/Jenkins/Bamboo, Python/Ruby/Scala/Bash/Go, ELK/AppDynamic, DynamoDB/MongoDB/ElasticCache/GridGain, Linux/Windows/AIX/Solaris, aPaaS/VMWare/Citrix,
Apr.,1996 – Jun., 2017 Earlier career USA,China Oracle 2014-2017 Lead Software Engineer Comcast 2010-2014 Sr. DevOps / Infra Automation Eng. Boomi 2006-2010 Sr. Software Engineer Vantage Learning 2006 Sr. System Engineer Manager Texas Tech University 2002-2006 Programmer/Analyst, Unix SysAdmin Sun Microsystems China 2000-2002 Sr. Software Engineer IBM China R&D 1998-2000 Software Engineer Taiji Computers 1996-1997 System / Network Engineer	- Oracle cloud integration connectivity SDK, Java connectors and Docker CI environments - Comcast AutoBuild: dependency-graph release automation for 200+ microservices - Boomi AtomSphere iPaaS: connector framework and production cloud platform on AWS - Kerberos/LDAP single sign-on, Globus grid computing and enterprise Java delivery Java/J2EE, SOA / EAI / iPaaS, Docker Swarm/Compose, CI/CD (Bamboo, Jenkins, Maven), Puppet/Capistrano, AWS EC2/S3, Linux HA clustering, JVM & GC tuning, Kerberos/LDAP/SSO,

SELECTED PROJECTS

1. Oct.,2025 – Current, Principal Cloud and Platform Architect (JPMC), Intuitive.ai , Cherry Hill, NJ (WFH),USA

Hands-on principal architect for the on-demand application platform used by hundreds of developers across more than 80 product teams. Own the ECS/EKS and Istio service-mesh reference architecture, the Terraform module library behind it, and the observability, secrets and TLS standards every onboarding service inherits. Cut new-service environment provisioning from weeks to hours and onboarded more than 200 microservices onto the paved road. Self-taught the firm's proprietary Environment as Code (EAC) platform and became the team's subject-matter expert, mentoring the team on it. Set the team's AI-assisted development practice, and packaged recurring platform work - service onboarding, environment triage, incident first-pass - as agent skills and subagents backed by an MCP server over the platform's own APIs.

1.1 On Demand Environment (ODE) backend (Java Spring Boot) infrastructure (ECS) design and implementation

Designed and implemented ODE to unify the provisioning and deployment flows for hundreds of applications, microservices and their supporting infrastructure, replacing per-team bespoke scripts with a single self-service paved road.

Highlights:

- Architected the backend infrastructure with ECS/ALB/R53/Fargate/RDS/IAM
- Designed and implemented reusable Terraform modules for different environments (Dev/Test/Prod)
- Built the Java Spring Boot backend that unifies provisioning and deployment behind one self-service flow, so a team requests an environment instead of assembling it
- Ran workloads on ECS Fargate to take EC2 capacity management out of the request path, with per-service autoscaling and rolling deployments in every environment
- Automated ALB listener and target-group wiring together with Route 53 records and ACM-issued certificates, so each provisioned environment comes up with a working TLS-terminated endpoint
- Provisioned RDS instances with per-service IAM task and execution roles, keeping database credentials out of application configuration and images
- Kept Dev/Test/Prod configuration DRY behind versioned module inputs, so an environment is reproduced from code rather than hand-assembled
- Made provisioning idempotent and repeatable end to end, taking new-environment turnaround from weeks of coordinated manual setup to hours
- Drove the team's AI-assisted development workflow on the ODE backend, drafting Spring Boot controllers, service layers and their unit tests with Claude Code and GitHub Copilot
- Turned the repetitive half of platform delivery into agent-driven tasks - service onboarding, Terraform module wiring, test scaffolding - each run against the platform's own conventions with an engineer approving the diff

Technologies:

Java, Spring Boot, AWS ECS, AWS Fargate, AWS ALB, Amazon Route 53, Amazon RDS, AWS IAM, AWS ACM, Terraform, Infrastructure as Code, Docker, CI/CD,

Leadership:

- Designed the On Demand Environment as a product with a single paved-road flow, consolidating the provisioning and deployment paths individual teams had each been maintaining themselves
- Set the reusable Terraform module contract that other platform engineers extended for new environment types
- Partnered with application teams during onboarding to fold their environment requirements back into the shared modules instead of one-off forks

1.2 Design and implement Splunk and Dynatrace onboarding for Spring Boot ECS Microservices

Designed and Implemented Splunk Universal Forwarder (UF) sidecar and Dynatrace OneAgent JVM instrument to collect application and JVM logs

Highlights:

- Architected the Splunk Universal Forwarder (UF) sidecar pattern for AWS ECS task definitions, adopted across all lower and production environments as the enterprise standard for container log routing
- Instrumented Java Spring Boot microservices with Dynatrace OneAgent injected at JVM startup via customized Dockerfiles, delivering APM, distributed tracing and JVM metric collection with no application code changes
- Built the reusable Terraform module that bootstraps every new service with the correct IAM roles, sidecar configuration and telemetry variables, cutting observability onboarding from days of hand-wired configuration to minutes
- Delivered Splunk dashboards and Dynatrace alerting profiles for application health and cross-service transaction latency, taking incident triage from hours of log-hunting to minutes of trace inspection
- Optimized Splunk log ingestion through logback.xml tuning, materially reducing indexed volume and the associated license cost
- Automated first-pass incident triage with an agent that pulls the failing service's Splunk logs and Dynatrace traces, correlates them across the call path, and hands the on-call engineer a written first hypothesis to confirm or discard

Technologies:

Splunk Universal Forwarder (UF), Dynatrace OneAgent, AWS ECS, Java, Spring Boot, Terraform, Docker, AWS IAM, Logback, APM / Observability,

Leadership:

- Standardized enterprise observability and telemetry practices by defining reusable, scalable patterns for logging and monitoring across all containerized AWS workloads
- Partnered with application development teams to troubleshoot complex performance issues using Dynatrace distributed tracing, significantly reducing Mean Time to Resolution (MTTR) for production incidents

1.3 Design and implement AWS Secrets (ASM) solution for Java Spring Boot Microservices

Architected and implemented a comprehensive AWS Secrets Manager solution to remove the secret value from the git repo for local development and aws deployment

Highlights:

- Designed and implemented the Terraform modules provisioning AWS Secrets Manager secrets, keys and values across every service and deployment environment

- Developed the Terraform resources creating least-privilege IAM read policies bound to ECS task roles, replacing broad shared policies with per-service access
- Integrated AWS SDK and Spring Cloud AWS in Java Spring Boot microservices for startup-time credential and configuration resolution, removing hardcoded secrets from the team's git repositories
- Established a secure local development workflow in which developers fetch secrets dynamically via temporary AWS credentials, eliminating local .env files from version control
- Enforced encryption at rest with AWS KMS and strict least-privilege access, closing the outstanding secrets-handling findings from security review

Technologies:

AWS Secrets Manager (ASM), Java, Spring Boot, Terraform, AWS ECS, AWS IAM, AWS KMS, Spring Cloud AWS,

Leadership:

- Championed DevSecOps best practices by successfully driving the initiative to eliminate hardcoded secrets across the organization's source code repositories
- Mentored backend engineering teams on secure credential management, IAM role assumptions, and AWS SDK integration for Spring Boot applications

1.4 EKS Service Mesh (Istio) on-demand framework Design and Implementation

Architected and implemented a comprehensive EKS-based service mesh framework using Istio to enable on-demand platform capabilities for hundreds of developers. Designed a scalable, secure, and automated infrastructure that supports dynamic service provisioning, traffic management, and observability across microservices.

Highlights:

- Designed and implemented the multi-tenant EKS + Istio architecture behind the on-demand platform, carrying the platform's full microservice estate across per-team tenant namespaces
- Developed the automation framework for dynamic service-mesh provisioning and configuration management, cutting tenant onboarding from weeks to under a day
- Enforced mTLS mesh-wide in STRICT mode for all service-to-service communication, replacing per-application TLS handling with a platform-level guarantee
- Configured Istio VirtualServices and DestinationRules for advanced traffic routing, load balancing and progressive rollout
- Designed and implemented centralized TLS/mTLS certificate management, eliminating manual per-service certificate renewal
- Built the mesh observability stack (Prometheus, Grafana, Jaeger, Kiali), giving product teams golden-signal dashboards and distributed traces they previously had to build themselves
- Authored the versioned Terraform modules for automated EKS and Istio deployment, consumed by product teams across the platform
- Implemented mesh policies for authorization, rate limiting and circuit breaking, containing retry storms and cascading failures at the mesh boundary rather than in application code
- Designed the multi-namespace isolation strategy for secure multi-tenancy, with ResourceQuota and LimitRange per tenant namespace and Istio AuthorizationPolicy enforced at each boundary
- Established CI/CD pipelines for service-mesh configuration changes, taking rollout from multi-day manual change windows to minutes behind automated lint, policy and staged-rollout gates
- Drafted and refactored mesh and Terraform configuration with Claude Code and Cursor - VirtualServices, DestinationRules, AuthorizationPolicies, module scaffolding - every generated change still gated by lint, policy checks and staged rollout

Technologies:

AWS EKS, Istio Service Mesh, Kubernetes, Terraform, TLS/mTLS, Envoy Proxy, Prometheus, Grafana, Jaeger, Kiali, Helm, AWS IAM/RBAC,

Leadership:

- Led the architecture and implementation of the service mesh framework that supports hundreds of developers on the on-demand platform.
- Established service mesh best practices and patterns that improved security, observability, and operational efficiency.
- Mentored platform and development teams on Istio service mesh concepts, traffic management, and security policies.

1.5 Agentic automation of platform operations: MCP server, agent skills and subagents

Automated the recurring operational work on the on-demand application platform - service onboarding, environment provisioning failures, IAM and TLS triage, incident first-pass - as reusable agent skills and subagents. An MCP server exposes the platform's environment and provisioning APIs to the coding agents that run them. Built on the firm's proprietary Environment as Code (EAC) platform, self-taught to subject-matter-expert level with no formal training or vendor support.

Highlights:

- Used AI coding assistants to reverse-engineer the undocumented EAC platform, summarizing its templates and mapping its abstractions onto the underlying AWS and Terraform resources, which cut the self-teaching curve sharply
- Set the team's working practice for AI-assisted development: assistants trusted for scaffolding, refactoring, test generation and codebase exploration; kept out of security-sensitive IAM and TLS changes; every generated change reviewed before it ships
- Built an MCP server exposing the platform's environment and provisioning APIs to coding agents, so an agent reads live environment state and drafts the matching EAC or Terraform change itself
- Packaged the team's recurring runbooks as reusable agent skills and subagents - provisioning failures, IAM and TLS triage, EAC template scaffolding - so diagnosis paths that lived in one engineer's head became workflows anyone can run

Technologies:

MCP, AI agents / subagents, Environment as Code (EAC), Terraform, AWS ECS, AWS EKS, AWS IAM, TLS / Certificate Management, CI/CD,

Leadership:

- Mentored the team on AI-assisted development - prompt and context habits, the review every generated change has to pass, and where the assistants are and are not trusted
- Moved the team from ad-hoc assistant use to recurring work packaged as agent skills and subagents, so the automation is shared and reviewable rather than rebuilt in each engineer's prompts

2. Feb.,2025 – Oct.,2025, Director of Cloud Architecture, Biophy, , Cherry Hill, NJ (WFH),USA

Hands-on director of cloud architecture for an AI-powered healthcare platform. Led the retrieval corpus pipeline behind the product's RAG system - multi-source ingestion, scraping, transformation and data-quality controls built on Python/Django/Pydantic with LangChain and the OpenAI API, structured under Clean Architecture and SOLID so the pipeline stayed testable as sources multiplied. In parallel brought the existing AWS estate under Terraform/Terragrunt by importing live resources and redesigning them to the Well-Architected Framework, and built a custom CloudWatch alerting system on EventBridge, Lambda, SQS and SNS.

2.1 Comprehensive Infrastructure as Code (IaC) Design and Implementation

Led the transformation of AWS infrastructure to infrastructure as code (IaC), importing and redesigning resources to align with best practices. Developed modular Terraform configurations and established secure, standardized cloud operations.

Highlights:

- Imported and catalogued existing AWS resources including EC2 instances, VPC configurations, security groups, and IAM policies
- Redesigned infrastructure architecture following AWS Well-Architected Framework principles
- Implemented modular Terraform configurations with reusable modules for consistent deployment
- Established comprehensive security controls including least-privilege IAM policies and network segmentation
- Created automated validation and testing procedures for infrastructure changes
- Documented infrastructure patterns and best practices for team adoption

Technologies:

Terraform, AWS EC2/EBS/S3/VPC/ALB/Security/R53/IAM/Secrets Manager, Infrastructure as Code, AWS Best Practices, Security Controls,

Leadership:

- Led the transformation from manual infrastructure management to fully automated IaC practices.
- Established infrastructure standards and patterns that improved team productivity and security posture.
- Mentored team members on AWS best practices and Terraform/Terragrunt development.

2.2 AI-Powered Data Ingestion and Scraping System for RAG

Architected and delivered a robust data ingestion and scraping platform for AI-powered RAG, applying SOLID and Clean Architecture principles to ensure scalability, maintainability, and high data quality.

Highlights:

- Architected a modular data ingestion pipeline supporting multiple data sources and formats
- Implemented intelligent web scraping capabilities with rate limiting and error handling across hundreds of recurring sources feeding the retrieval corpus
- Built the chunking, embedding and upsert path that turned heterogeneous scraped and uploaded documents into a queryable corpus of tens of thousands of embedded chunks, with deduplication and re-embedding on source change
- Designed data transformation layers following Clean Architecture principles
- Built robust error handling and retry mechanisms for data processing failures
- Implemented comprehensive logging and monitoring for data pipeline health
- Created automated testing frameworks for data quality and pipeline reliability

Technologies:

Python/Django/Pydantic, openAI API/LangChain, SOLID Principles, Clean Architecture, Data Processing, Web Scraping,

Leadership:

- Drove the adoption of Clean Architecture and SOLID principles across the data engineering team.
- Established data quality standards and monitoring practices for AI system reliability.
- Mentored developers on modern software design patterns and best practices.

3. Dec.,2021 – Feb.,2025, Senior Cloud Engineer II (Associate), McKinsey & Company , Cherry Hill, NJ (WFH),USA

Built GenAI platform capability for enterprise modernization engagements: retrieval systems over ChromaDB/Milvus, multi-agent orchestration with AutoGen, and FastAPI services that exposed model-backed capabilities as internal APIs other engineering teams consumed rather than rebuilt. Scaled the retrieval layer behind a call-centre chatbot copilot from a single-node prototype to a persistent, concurrently queried vector store as the knowledge corpus and user base grew. Led the GenAI + RAG system that accelerated mainframe migration to modern stacks, the RAG-assisted code generation platform for PySpark/Databricks data pipelines, and the scaling of the retrieval layer behind a call-centre chatbot copilot. Separately migrated and refactored complex mainframe event-driven systems onto AWS serverless (EventBridge, Lambda, Step Functions, SQS/SNS, CloudWatch, EKS).

3.1 Using genAI (openAI) with RAG (ChromaDB, Milvus) to accelerate the development of data pipelines running on pyspark and databricks

Designed and implemented a genAI system with RAG (ChromaDB/Milvus) to automate and accelerate the development of data pipelines, enabling seamless transformation of source tables to target tables with varying data structures.

Highlights:

- Prompt engineering and tuning to generate PySpark data-pipeline functions together with their unit tests
- Designed the context-window management strategy - chunking by target table or by relationship - so large schema inputs fit within model token limits without losing referential context
- Built concurrent inference orchestration over the OpenAI API using asyncio with generators, raising pipeline throughput without exceeding rate limits
- Curated the retrieval corpus from the enterprise data dictionary - source and target schemas, column-level lineage and prior transformation examples - embedded with OpenAI embeddings into HNSW-indexed vector collections spanning thousands of table definitions
- Tuned the retrieval layer rather than only the prompt: chunk granularity, top-k depth and similarity thresholds, measured against whether the generated PySpark compiled and passed its generated tests, which mattered more to output quality than further prompt iteration
- Operated the vector store as shared infrastructure - collection layout, embedding refresh on schema change, and reproducible rebuilds from source - so retrieval quality did not silently decay as the data dictionary evolved, holding sub-second retrieval at interactive latencies
- Implement FastAPI API to expose source to target data transformation as REST service
- Design and document the local development and debug workflow by using debugpy with VSC and pycharm

- Paired the generation service with IDE-level assistance (GitHub Copilot, later Cursor): the RAG service generated schema-aware pipelines, in-editor completion handled the surrounding code and tests
- Closed the generate-test-repair loop: a failing unit test fed straight back into the next generation attempt, so the pipeline converged on compiling, passing PySpark without hand re-prompting

Technologies:

Langchain/VectorStore/ChromaDB/Milvus, genAI/openAI, RAG, Python/Pydantic/Asyncio/Flask/FastAPI/Debugpy, Typescript/JavaScript/NodeJs, Microservice, Dockerfile/Docker/Compose, PostgreSQL,

Leadership:

- Spearheaded the integration of genAI and RAG for data pipeline automation, enabling rapid transformation of complex data structures.
- Mentored engineers on advanced prompt engineering and parallel AI workflow design, raising team proficiency in cutting-edge AI techniques.
- Established robust local development and debugging standards, improving team productivity and onboarding efficiency.

3.2 Using genAI (openAI) with multi-agents to accelerate the migration of mainframe to modern technologies

Built a multi-agent genAI platform to automate and accelerate the migration of mainframe applications to modern technologies, integrating RAG for context-aware transformation and process automation.

Highlights:

- Core Contributors to genAI system to migrate mainframe frontend technologies (BMS/JSF/JSP) to modern technologies (React/Angular..etc.)
- Contribute to develop multi-agents genAI system to automate the development/testing/deployment process
- Built the RAG layer that accelerated COBOL-to-Java migration, operating ChromaDB/Milvus as shared retrieval infrastructure holding the dependency context that multiple migration workstreams queried
- Indexed the mainframe dependency graph - copybooks, JCL, called modules and data definitions across a legacy estate of thousands of COBOL programs - so an agent converting one module retrieved its true upstream and downstream context instead of guessing at it
- Sized and partitioned collections per application domain to keep retrieval relevant as the corpus grew, running ChromaDB for fast local iteration and Milvus where persistence and concurrent multi-agent querying were needed
- Designed the human-in-the-loop step around the agents: engineers reviewed, corrected and completed the generated COBOL-to-Java and BMS-to-React output with in-editor assistance (GitHub Copilot, Cursor) before any of it landed in the codebase
- Designed the agent workflow itself - analysis, conversion, test-generation and review agents, the hand-off contract between them, and the retry and escalation path when output failed its tests - so a migration ran as a repeatable pipeline rather than a string of manual prompts

Technologies:

Langchain/VectorStore/ChromaDB/Milvus, genAI/openAI, RAG, Agents/Multi-agents/Autogen, Python/FastAPI, Typescript/JavaScript/NodeJs, Microservice, Dockerfile/Docker Compose, PostgreSQL,

Leadership:

- Drove the integration of multi-agent genAI systems to automate legacy migration, reducing manual effort and error rates.
- Collaborated across frontend and backend teams to ensure seamless modernization from mainframe to cloud-native stacks.
- Championed the use of RAG for context-aware migration, setting a new benchmark for future modernization projects.

3.3 Migration/Refactoring of Mainframe (IBM MQ) EDA system to AWS by using EventBridge/Lambda/Step Function/CloudWatch/EKS/SQS/SNS...etc.

Engineered and deployed an EKS-based router to integrate AWS EventBridge with external MQ events, providing robust monitoring, reporting, and cloud-native migration for mainframe EDA systems.

Highlights:

- Develop SpringBoot EKS router to convert XML/XSD to JSON schema and integrate with MQ events
- Design and implement the monitoring terraform module for all different AWS resources (Lambda, EventBridge, S3, DynamoDB, Step Function..etc.) using cloudwatch metrics
- Design and implement the reusable report generator using Lambda and EventBridge and deployed as Microservices which can be called to run any custom AWS CloudWatch insights queries in custom schedule

Technologies:

Java/SpringBoot, Terraform, Microservice, Java/JavaScript, EKS, AWS/EventBridge/CloudWatch/Lambda, XML/XSD/Json/Json Schema, DynamoDB, RDS/Oracle,

Leadership:

- Led the end-to-end migration of mission-critical EDA systems to AWS, ensuring zero downtime and robust cloud integration.
- Developed reusable infrastructure modules, empowering other teams to accelerate their own cloud migration journeys.
- Instituted comprehensive monitoring and reporting standards, elevating operational visibility and reliability.

4. Dec.,2020 – Dec., 2021, Staff Engineer, Yotascale , Cherry Hill, NJ (WFH),USA

Lead architect and lead implementation engineer for a multi-cloud (AWS/Azure/GCP) cost analytics, optimization and management platform - the FinOps tooling layer giving customers unit-cost visibility, allocation/showback and rightsizing recommendations across enterprise customers' multi-account cloud estates and the large annual cloud spend under management. Owned both the product architecture (design-first microservices on ClickHouse/PostgreSQL) and the AWS platform it ran on (EKS/Istio/Terraform/Terragrunt).

4.1 Lead Architect and Lead Implementation Engineer for Cloud(AWS/Azure/GCP) Cost Analytics/Optimization/Management platform

Architected and implemented a cross-cloud cost analytics and optimization platform, enabling organizations to gain actionable insights and manage cloud spending across AWS, Azure, and GCP. Led the design of scalable data pipelines, analytics microservices, and secure infrastructure.

Highlights:

- Introduce and Build Microservice Design First Methodology and team culture based on OpenAPI/Swagger Specification
- Design and develop Cost Analytics/Lens Navigation/Lens Management Microservice using SpringBoot with ClickHouse and PostgreSQL as DW and DBMS
- Design and implement dynamic metadata driven widget/filter configuration Microservice
- Implement bitbucket CI/CD pipeline to build, deploy Microservices/GraphQL to eks/k8s with Istio

- Design the AWS infrastructure architecture with EKS/ALB/Istio/S3/SES/R53/ECR/PostgreSQL/RabbitMQ/SFTP
- Design/Implement terraform templates/scripts to deploy the full AWS infrastructure stack with IAM roles/users
- Develop terraform templates to deploy ALB controller/RabbitMQ and PostgreSQL by using helm provider
- Develop terraform templates to deploy k8s' CRDs, Pods, Services, Service Account by using Kubernetes-alpha and Kubernetes providers
- Design/Develop multiple terraform modules with Terragrunt.hcl to maintain the inputs/outputs dependencies between modules and keep configurations DRY
- Develop terraform templates to dynamically add R53 zone and records with TLS certificates validations
- Develop terraform templates to dynamically create S3 buckets with IAM policies to allow k8's service account to assume IAM role to access control using OIDC
- Develop terraform templates to create SFTP server backed with S3 bucket with IAM role and policy for sftp users
- Compile Microservice design/development guide lines based on OpenAPI/Swagger Spec
- Implement Junit test cases with local docker test containers
- Develop common multiple DBMS (ClickHouse and PostgreSQL) access lib with AWS secrets manager
- Implement json schema validation using open source
- Delivered cost allocation, showback and rightsizing views over multi-terabyte cloud billing datasets in ClickHouse, holding interactive query latency on high-cardinality, multi-dimension slices
- Established the FinOps model inside the product - tagging/allocation taxonomy, unit-cost metrics, savings-plan and rightsizing recommendations - surfacing double-digit percentage savings opportunities for customers
- Stood up the entire AWS platform as code with versioned Terraform/Terragrunt modules, provisioning a complete environment from an empty AWS account in hours rather than weeks
- Onboarded the engineering teams onto the design-first OpenAPI workflow, making the published contract the single source of truth for client and server code

Technologies:

Cloud Cost Analytics/Optimization/Management, OpenAPI/Swagger, Microservice, Java/Kotlin/Python, Spring/SpringBoot/Flask, Kubernetes/EKS/AKS/GKE, Terraform/Terragrunt/Ansible, AWS/EC2/EBS/S3/VPC/Security/R53/IAM/Secrets Manager/Lambda, Service Mesh/Istio/Envoy, Helm/Chart, RabbitMQ, Azure/GCP, Junit/Test Containers, Docker/Minikube,

5. Mar.,2020 – Nov., 2020, Technical Infrastructure/Devops Architect, Gigster Inc. , Cherry Hill, NJ,USA

Designed and implemented comprehensive AWS infrastructure architecture including CI/CD automation, Terraform infrastructure as code, and EKS container orchestration. Led the development of scalable, automated cloud infrastructure solutions to support microservices and distributed systems.

5.1 AWS EKS platform for Node.js microservices and event-driven workloads

AWS architecture for Node.js microservices and event-driven workloads on EKS, fully provisioned through Terraform/Terragrunt with ALB ingress controller, IAM/OIDC service-account roles and Helm-deployed dependencies.

6. Jun.,2017 – Feb., 2020, VP,Lead Architect Engineer, Barclays , Wilmington, DE,USA

Directed the cloud adoption journey, migrating on-premises enterprise systems to AWS. Served as delivery lead for distributed system technology, including microservices, event-driven architecture, message transport, integration (batch/ETL), and event stream processing.

6.1 Delivery Lead for strategic distributed system technology domain

Directed the architecture and technology strategy for distributed systems, microservices, event-driven architecture, and stream processing, driving innovation and technology transformation across the organization.

Highlights:

- Owned end-to-end architecture for the group's distributed-systems domain: solution-architecture guidance, design-pattern development, and architecture reviews across the domain's application estate
- Ran architecture reviews identifying reuse opportunities and managing technical debt, with documented invest/divest and reuse decisions feeding the domain roadmap
- Maintained the domain capability roadmap and technology heatmap, monitoring emerging technologies and industry trends for enterprise applicability
- Influenced group-level strategic planning and drove innovation forums across the group's engineering teams
- Drive technology transformation from on-premises to public cloud, multi-tiers monolithic to cloud-native Microservice system, strong ACID to BASE data consistency, Sync/Request/RPC driven to Async/Event driven architecture, tight coupling to loose coupling, ETL/Batch/Time to realtime streaming processing, vendor locked to resilient commodity system, vendor favor to OSS, vertical scaling to horizontal scaling
- Establish group-wide distributed system architecture and technology strategy
- Build PoC and Reference Architecture for common use cases for distributed system
- Evaluate around 200 existing products/technologies and decide the invest/divest house positions
- Create the architecture handbook for Microservice as group-wide guidance for solution Architect
- Contribute to and review the architecture handbook for EDA, Stream Processing and Integration
- Design vendor and technology selection and design principles/standards and best practices for the domain
- Review and approve the new request for product/technology in the domain
- Maintain the scope document and product list for the domain
- Develop the roadmap to adopt new products/technology and decommission legacy products/technology
- Lead working group (~30 members) forum to discuss and make decision on technology strategies

Technologies:

Technology strategic Design and Stewardship, Distributed system, Domain Driven Design, Microservice, ACID/BASE, (Event) Stream /Batch Processing, Event Driven Architecture (EDA), Message Transports (Kafka, ActiveMQ, IBM MQ), Data Integration (ETL/ELT), Compensating transactions/Sagas, JMS/AMQP, Resilience and Performance, CQRS/SOA, CAP,

Leadership:

- Chaired a ~30-member cross-functional working group that set and executed the group-wide distributed-systems technology strategy, issuing invest/divest house positions on ~200 products and technologies
- Authored the group-wide Microservices Architecture Handbook and contributed to the EDA, Stream Processing and Integration handbooks - reference artifacts used group-wide by Barclays solution architects
- Owned cloud-adoption strategy and roadmap for Contact Center, Merchant Services and Fraud: assessed ~500 on-premises applications for migration readiness and guided the prioritized migration waves to AWS
- Drove the transformation agenda - on-premises to public cloud, monolith to cloud-native microservices, ACID to BASE, synchronous RPC to asynchronous event-driven, batch/ETL to real-time streaming, vendor-locked to OSS commodity, vertical to horizontal scaling

6.2 Group-wide Cloud Adoption for Contact Center, Merchant Services and Fraud

Led cloud assessment and migration strategy for hundreds of on-premises applications, providing technical guidance and troubleshooting to ensure successful adoption of AWS cloud architecture.

Highlights:

- Build python automation tools to access AWS secret manager via boto3
- Using python boto3 to build tools to dynamically manipulate the IAM policies, roles and users
- Using terraform to dynamically deploy whole AWS infrastructure including EKS, IAM, DNS, ALB, S3 ...etc.
- Building and maintaining the EKS cluster via terraform and k8s yaml
- Defined and designed cloud strategy and roadmap/journey for Merchant services, Fraud and Contact Center
- Ensured ~500 on-premises application cloud migration journey
- Assess ~500 on-premises applications to determine the cloud migration readiness
- Provide guidance to application team to design cloud migration architecture
- Guide the .NET system to AWS migration
- Consulting to migration on-premises system to AWS ECS
- Collect system data from Abacus, ServiceFirst query and repo tools

Technologies:

AWS, EC2, RDS, S3, ECS, VPC, ACL/Security Group/Routing table, ELB/NLB/ALB, Private Link, EBS,

Leadership:

- Led the assessment and migration of 500+ on-premises applications to AWS, ensuring business continuity.
- Developed cloud migration roadmaps and provided hands-on guidance to application teams.
- Championed automation of cloud infrastructure, reducing manual errors and accelerating delivery.

7. Apr.,1996 – Jun., 2017, Lead Software Engineer / Senior DevOps & Infrastructure Engineer / Systems Engineer, Earlier career , USA, China

Twenty-one years of platform, automation and integration engineering preceding the current cloud-native and GenAI work. At Oracle, built the cloud integration connectivity SDK and Java connectors joining SOA Suite and Integration Cloud Service to SaaS and on-premises applications, and pioneered Docker Swarm/Compose CI environments for the connector teams. At Comcast, designed the AutoBuild system that built and released 200+ microservice projects from a dependency graph, cutting release cycles from days to hours, and ran the Gerrit/Bamboo/Nexus estate on Pacemaker/HAProxy clusters with Puppet and Capistrano automation. Earlier: the SOA connector framework and production cloud platform at Boomi - one of the first iPaaS products - on AWS EC2/S3 and managed data centres, including JVM/GC tuning and Apache Solr indexing at millions of transactions per day; a 100+ server data centre at Vantage Learning; Kerberos/LDAP single sign-on and Globus grid computing at Texas Tech University alongside an MS in Computer Science; and enterprise Java, payments and storage-management systems at Sun Microsystems China and IBM China R&D.

7.1 Oracle cloud integration connectivity SDK, Java connectors and Docker CI environments

SDK and plugin framework linking Oracle SOA Suite and Integration Cloud Service to SaaS and on-premises enterprise applications, with Docker Swarm/Compose/Stack CI test environments built for geographically distributed connector teams.

7.2 Comcast AutoBuild: dependency-graph release automation for 200+ microservices

Directed-acyclic-graph driven build, branch and release automation across 200+ repositories, reducing release cycles from days to hours, with the Gerrit/Bamboo/Nexus estate on Pacemaker/HAProxy clusters and configuration managed by Puppet and Capistrano.

7.3 Boomi AtomSphere iPaaS: connector framework and production cloud platform on AWS

SOA connector framework with synchronous/asynchronous messaging, plus the clustered production platform on EC2/S3 including distributed caching, Solr indexing at millions of transactions per day and JVM heap/GC tuning.

7.4 Kerberos/LDAP single sign-on, Globus grid computing and enterprise Java delivery

Cross-realm Kerberos KDC, OpenLDAP and PAM single sign-on across Unix and Windows at Texas Tech University; OGSA/OGSI grid deployment with PKI/X.509 security; J2EE architecture at Sun Microsystems China and IBM China R&D.